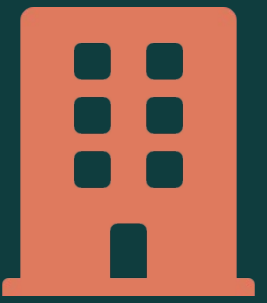
How much is this apartment really worth?



Predicting the asking price of apartments in Astana and Almaty from krisha.kz listings

Course project, Midterm · Machine Learning Algorithms · Direction 1 (Apartments for sale) · Regression

Rakhat Balgabekov, Aitbek Kaiyrbekuly · Group SE-2424 · October 2026

Question: can a listing's fields tell us a fair asking price?

How accurately can we predict the asking sale price (KZT) of an apartment in Astana or Almaty from its listing characteristics: total area, rooms, floor and total floors, building type, year built, district, condition and other attributes?

Asking prices are set by sellers and often differ from what similar apartments go for. A model gives an independent reference.

Main metric

- **MAE in KZT**
- the error in the money the user thinks in; robust to a few luxury ads
- **Median absolute % error (MdAPE)**
- our registration goal is a median error under 10 %
- RMSE and R^2 reported for completeness

Who uses the answer



Buyer

Is this asking price fair before I spend an evening viewing it?



Seller

A realistic starting price so the flat does not sit unsold for months



Realtor

A quick reference estimate for every new listing

Data: 4 815 krisha.kz listings, collected with our own scraper on 4–8 Oct 2026

5 810
ad cards from the search pages, Astana and Almaty

4 802
full ad pages opened (structured fields, text, photos)

4 815
rows after cleaning: 2 503 Astana, 2 312 Almaty



Stage 1

Search pages (20 ads each) → every ad card: id, price, title, district, preview



Stage 2

Mobile ad page → building type, year, floors, kitchen, condition, parking, coordinates, photos, description



Source check

robots.txt allows these pages, no login, objects not people: seller name and phone are not stored

Ad pages collected in two halves (4 and 8 Oct): the site's bot challenge paused new sessions in between and we waited rather than working around it; 13 ads removed meanwhile fall back to their search card (flag has_detail).

One row = one ad, 23 fields

- Target: asking price, KZT
- Size: area, rooms, kitchen area, floor / total floors
- Building: type, year built, ceiling height, residential complex
- State: condition, bathroom, furnished, balcony, parking, mortgaged
- Place: city, district, coordinates
- Seller: owner / agent, photo count, urgent label
- Text for the Final: seller's description, photo URLs

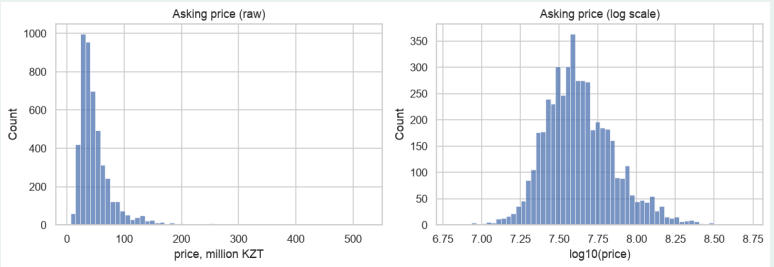
Preparation: every cleaning step has a reason and a before/after count

Step	Why	Before	After
numeric fixed price	no target: developer 'from' prices	4,822	4,822
unique listing id	'hot' block repeats ads across pages	4,822	4,822
remove near-duplicate reposts	agents repost the same flat under a new id	4,822	4,817
plausible ranges	typos and test ads would dominate RMSE	4,817	4,815
kitchen/ceiling set to NaN	kitchen ≥ total area, ceiling outside 2.2–5 m	4,815	4,815

Leakage check: what the model must not see

- price per m² (computed from the target), price text, title and preview
- description: Final-stage source, often states the price in words
- ids, timestamps, promotion status: not properties of the apartment
- kept: only what a seller fills in when the ad is created
- imputation and scaling live inside the pipeline → fitted on the training fold only

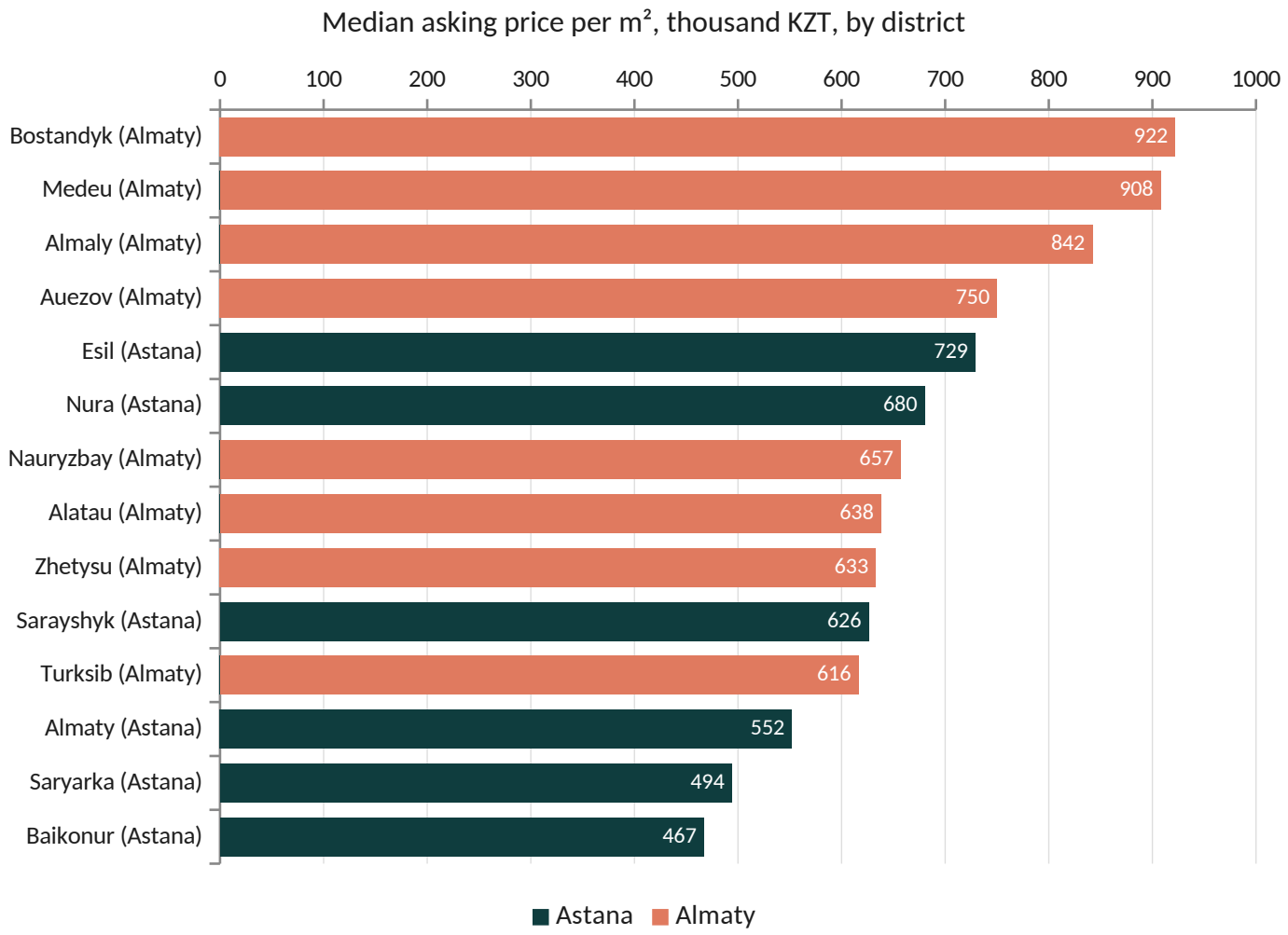
Target modelled as log(price)



Raw prices are strongly right-skewed: the median ad is 40 M KZT, the top 5 % are above 115 M. A linear model on raw prices would spend its capacity on a few luxury ads.

Decision: fit on log(price), transform back to KZT, compute every metric in KZT. Area enters as log(area) too, so price ∝ area^β instead of e^area.

Exploration: location sets the price level, area sets the scale, age bends it



District changes price per m² almost 2×; Almaty costs more at the same rank

→ city and district stay as features; the district rule becomes our baseline

Price vs area is a straight band on the log scale, but 3× wide at 60 m²

→ log target, log(area); non-linear models needed for the rest

Price per m² vs year built is U-shaped: Soviet brick cheap, 2000s middle, post-2018 dearest; panel houses cheapest

→ year as a number + building type as a category

Area, rooms and kitchen area are strongly correlated

→ Ridge and Lasso in the comparison

Modelling workflow: fixed split, pipeline, two baselines, five models, 5-fold CV

1 · Fixed split

80 / 20, random_state 42; test ids saved and reused at the Endterm and the Final

2 · Pipeline

median impute + missing flag + StandardScaler; one-hot with rare levels grouped; fitted on the training fold only

3 · Baselines

dummy median, and the realtor's rule: district median price per m² × area

4 · Models + CV

LinReg, Ridge, Lasso, KNN, Decision Tree; same 5 folds for all; test set touched once

Why each model is in the comparison

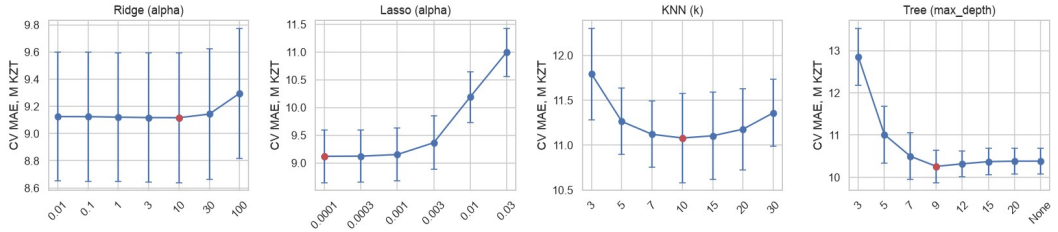
Linear Regression: simplest explainable model; plot 3 showed a log-linear relation

Ridge / Lasso: correlated features (area, rooms, kitchen); Lasso drops useless one-hot levels

KNN (distance-weighted): 'similar flats nearby cost similar money'; scaling guaranteed by the pipeline

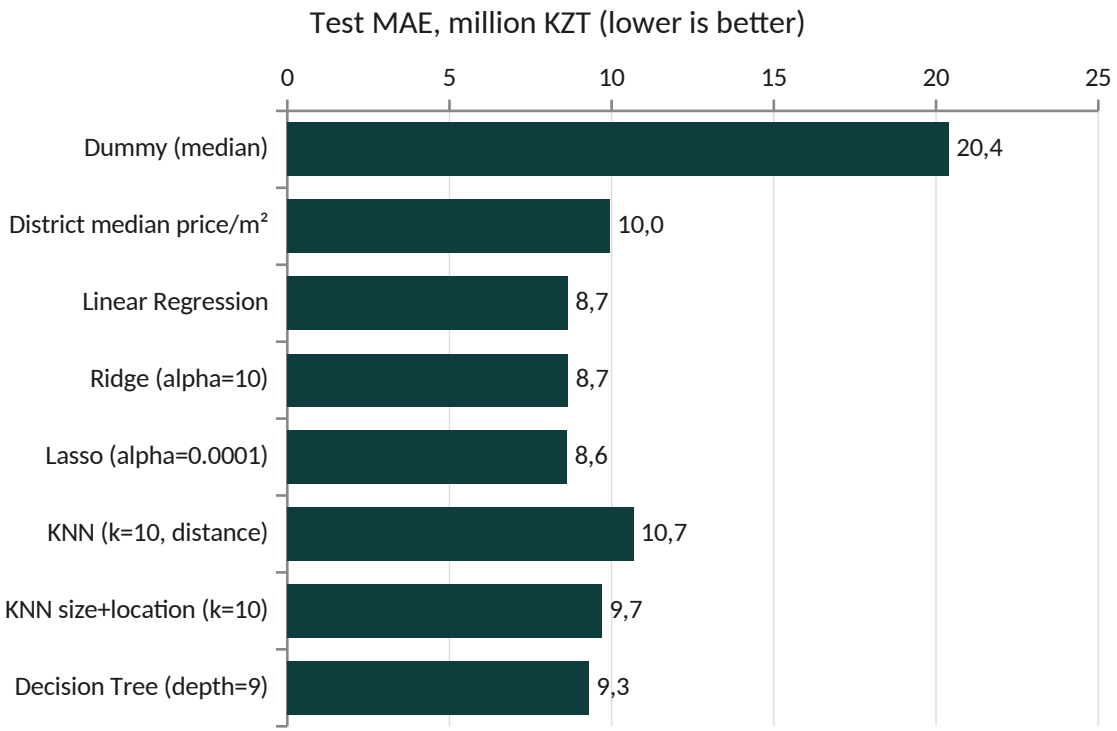
Decision Tree: captures the U-shaped year effect and interactions without feature engineering

Main hyperparameters chosen by CV MAE, not by default



10	0.0001	10	9
Ridge α	Lasso α	KNN k	Tree depth

Results: a linear model on log price beats the realtor's rule by 1.3 M KZT per ad



Model	CV MAE	CV MdAPE %	Test MAE	Test MdAPE %	Test R^2
Dummy (median)	20.80 ± 0.62	32.2 ± 1.5	20.4	33.1	-0.07
District median price/m²	10.47 ± 0.53	14.6 ± 0.5	9.9	14.3	0.69
Linear Regression	9.12 ± 0.47	12.1 ± 0.4	8.7	11.3	0.72
Ridge (alpha=10)	9.12 ± 0.48	12.0 ± 0.3	8.7	11.4	0.71
Lasso (alpha=0.0001)	**9.12 ± 0.47**	**12.1 ± 0.4**	**8.6**	**11.3**	**0.72**
KNN (k=10, distance)	11.08 ± 0.50	15.8 ± 0.8	10.7	16.1	0.67
KNN size+location (k=10)	10.14 ± 0.63	13.7 ± 0.2	9.7	13.4	0.72
Decision Tree (depth=9)	10.26 ± 0.38	13.4 ± 0.4	9.3	12.7	0.75

CV = 5-fold cross-validation on the training set, mean ± std, in million KZT. Test = the held-out 20 %, touched once.

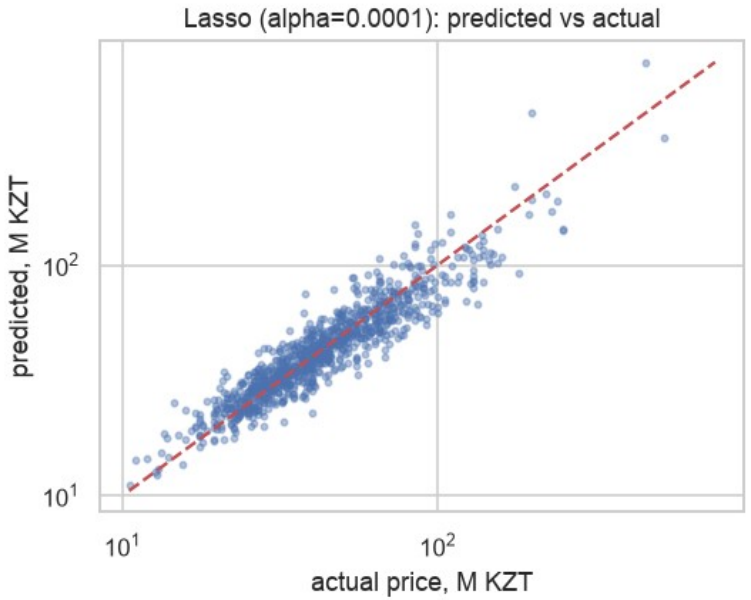
8.6 M
test MAE of the best model, KZT

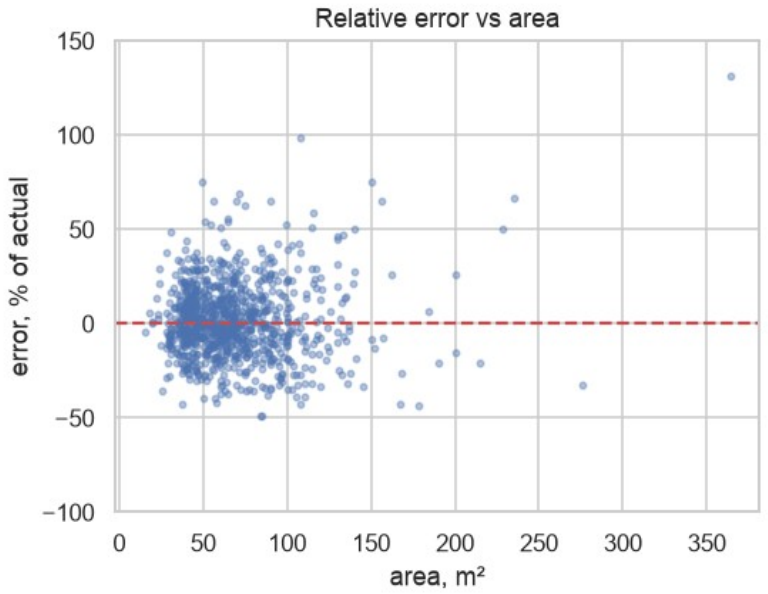
11.3 %
median relative error (goal < 10 %)

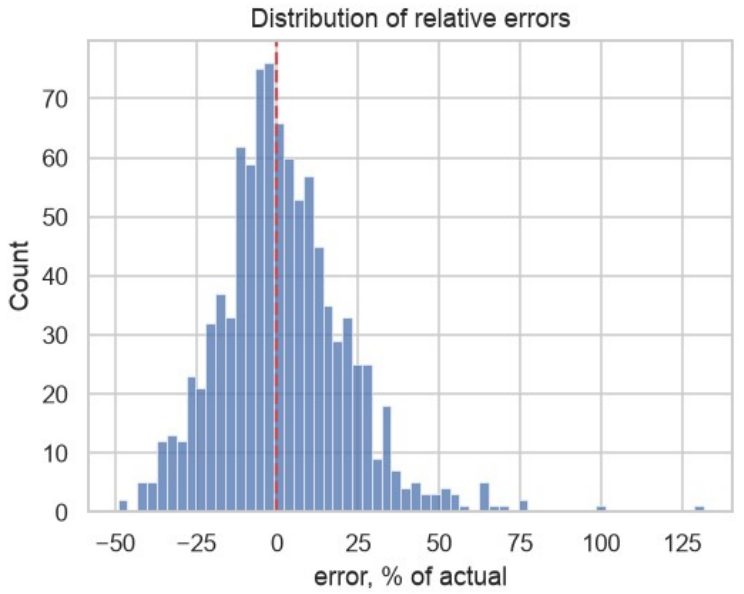
73 %
of test ads within ±20 % of the prediction

The three linear models tie. KNN on size + location only: MdAPE 13.4 % vs 16.1 % on all columns — the distance must compare the right things. The tree comes closest and wins on R^2, so ensembles of trees are the next step.

Where the model fails: expensive flats, big flats, and ads that are cheap for a reason only in the text







By price band

MdAPE about 10 % for 20–50 M KZT, 21 % above 80 M, about 30 % above 150 M: few training ads and unobserved quality (view, finishing, the exact house)

By size

1-room flats best (≈10 %), 4+ rooms worst; errors widen above ~120 m², where ads are rare and prices depend on finishing and view

Worst cases

'cash only', 'documents at the end of the year', assignment sales in 2025–26 buildings, a townhouse listed as a flat: the fields describe a normal new flat, the text explains the discount

Conclusions and Endterm plan: every next step answers a measured weakness

Answer to the question

Structured fields alone give a median error of 11.3 % (MAE 8.6 M KZT), better than the realtor's rule (14.3 %), not yet under the 10 % goal. A buyer can flag any ad priced more than ~20 % above the estimate; a seller gets a defensible starting price.

Limitations

- asking price ≠ sale price: the seller's premium is invisible to us
- a snapshot of one week (4–8 Oct 2026): prices move with rates and mortgage programmes
- seller-typed fields: wrong condition, year or area in a noticeable share of ads
- kitchen area is missing in most ads; the luxury tail (> 150 M) is thin



Ensembles + tuning

Weakness: a single tree already wins on R^2 but is unstable across folds and fits a staircase.

Random Forest and Gradient Boosting, randomised search inside the same 5-fold CV. Target: MdAPE < 10 %.



Clustering + PCA

Weakness: errors grow with price; one global model treats the luxury segment like the mass market.

K-Means / DBSCAN on area, rooms, year, price per m², coordinates → segment as a feature; PCA for a compact KNN space.



Neural network

Weakness: the linear model cannot bend, the tree bends only in steps.

MLP regressor on the scaled features, compared with boosting on the same saved split.

Same saved split (data/clean/split_ids.csv) → Endterm numbers directly comparable with today's table. Description text and photos are added at the Final.